# BTech Exam Question Bank

**Unit 1.pdf**
PDF

**Unit 2.pdf**
PDF

**Unit 3.pdf**
PDF

prepare a lot btech exam level questions from these PDFs. I need it to prepare for my exams.

Got it. Straight to the point. Below is a **large, BTech exam-oriented question bank** strictly derived from **Unit 1, Unit 2, and Unit 3 PDFs** you uploaded. These are framed exactly the way **university examiners ask**: definitions, comparisons, explain-with-diagram, short notes, numericals, and scenario questions.

I have grouped them **unit-wise and mark-wise** so you can revise smartly.

All content is taken from your PDFs

---

## UNIT I – Agile Fundamentals & Manifesto

## 2-Mark Questions (Very Short)

1. Define software development.
2. What is Agile Software Development?
3. Mention any two drawbacks of the Waterfall model.
4. What is meant by incremental development?
5. State any two Agile values.
6. What is a sprint?
7. What is meant by “responding to change” in Agile?
8. Define stakeholder.
9. What is the Agile Manifesto?
10. What is RAD?

## 5-Mark Questions (Short Answer)

1. Explain the need for Agile Software Development.
2. Compare traditional software development and Agile development.
3. Explain the four Agile values with examples.
4. Describe the concept of incremental delivery in Agile.
5. Explain the onion model of stakeholders.
6. What challenges are faced while implementing Agile?

7. Explain the role of customer feedback in Agile.
8. Describe sequential (plan-driven) development models.
9. Explain why documentation is given less importance in Agile.
10. Write short notes on:
    - Waterfall model
    - Spiral model
    - Incremental model

## 10-Mark Questions (Long Answer)

1. Explain Agile Software Development in detail with a real-world example.
2. Describe the Agile Manifesto and explain all four values.
3. Explain the 12 principles of Agile Software Development.
4. Compare Agile and Waterfall models with suitable examples.
5. Explain stakeholder classification using the onion model with a diagram.
6. Discuss the limitations of traditional development models and how Agile solves them.

---

# UNIT II – Agile Frameworks, Scrum, XP, Lean

## 2-Mark Questions

1. What is Scrum?
2. Define Product Backlog.
3. Who is a Product Owner?
4. What is a Scrum Master?
5. What is Kanban?
6. Define WIP limits.
7. What is Kaizen?
8. What is XP?
9. What is a Sprint Review?
10. What is Definition of Done?

## 5-Mark Questions

1. Explain Scrum roles.
2. Describe Scrum ceremonies.
3. Explain Scrum practices.

4. What are the advantages of Scrum?
5. Explain Lean Software Development.
6. Explain Kanban with a workflow diagram.
7. Describe the five Scrum values.
8. Explain Extreme Programming and its philosophy.
9. Write short notes on:
    - Pair programming
    - Test Driven Development
10. When should Scrum be used?

## 10-Mark Questions

1. Explain Scrum framework in detail with roles, ceremonies, and artifacts.
2. Describe how Scrum works step by step.
3. Compare Scrum and Kanban.
4. Explain Lean principles and waste management in Agile.
5. Explain Extreme Programming and its 12 practices.
6. Compare Agile model with RAD model.
7. Compare Agile model with Iterative Waterfall model.
8. Explain scaling Agile using SAFe, LeSS, and Scrum of Scrums.

---

# UNIT III – Agile Planning, Estimation & Metrics

## 2-Mark Questions

1. What is Agile estimation?
2. Define velocity.
3. What is sprint burndown chart?
4. Define cycle time.
5. What is throughput?
6. What is escape rate?
7. Define ROI.
8. What is lead time?
9. What is defect density?
10. What is time-to-market?

## 5-Mark Questions

1. Explain Agile metrics and their importance.
2. Describe team-level Agile metrics.
3. Describe product-level Agile metrics.
4. Describe process-level Agile metrics.
5. Explain Agile planning levels.
6. Explain sprint-level planning and estimation.
7. Explain release-level planning.
8. What is Planning Poker?
9. Explain Wideband Delphi estimation technique.
10. Write short notes on:
    - Burndown chart
    - Velocity

## 10-Mark Questions

1. Explain Agile planning and estimation in detail.
2. Describe different Agile metrics with examples.
3. Explain project-level, release-level, and sprint-level planning.
4. Discuss estimation techniques used in Agile projects.
5. Explain the importance of Agile metrics in monitoring progress.
6. Explain how velocity is calculated and used for planning.

---

## NUMERICAL / PROBLEM-BASED QUESTIONS (Very Important)

1. A team completes 15, 25, 30, and 20 story points in four sprints. Calculate velocity.
2. A sprint starts with 40 story points and 27 are completed. Find remaining work.
3. A team commits to 20 stories but completes 17. Calculate escape rate.
4. A task starts on Day 5 and finishes on Day 11. Find cycle time.
5. A project costs ₹1,00,000 and delivers value ₹1,40,000. Calculate ROI.
6. A system has 20 defects in 5 modules. Find defect density.
7. Stories with values 20, 30, and 40 are completed. Find value delivered.
8. A request is made on Day 7 and delivered on Day 21. Find lead time.
9. Development starts on 10th Jan and release is on 25th Feb. Find time-to-market.
10. A team completes 18 backlog items in a sprint. Find throughput.

---

If you want, next I can:

- Convert this into **model answers**
- Highlight **most probable exam questions**
- Create **1-day / 3-day / 7-day revision plan**
- Generate **unit-wise MCQs**
- Create **previous-year-style question papers**

Tell me how hard you want to go.

give me all the questions along with their complete answers:
Define software development.

What is Agile Software Development?

Mention any two drawbacks of the Waterfall model.

What is meant by incremental development?

State any two Agile values.

What is a sprint?

What is meant by “responding to change” in Agile?

Define stakeholder.

What is the Agile Manifesto?

What is RAD?

Below are **clean, exam-ready answers**, written exactly at **BTech 2-mark level**. No fluff. You can reproduce these verbatim in exams.

All answers are strictly based on your course PDFs

Unit 1

## 1. Define software development.

**Software development** is the process of **designing, creating, testing, and maintaining computer programs and applications** to perform specific tasks and meet user needs.

## 2. What is Agile Software Development?

**Agile Software Development** is a **flexible, iterative, and incremental software development methodology** that emphasizes **customer collaboration, working software, continuous feedback,**

**and the ability to adapt to change** throughout the development process.

---

### 3. Mention any two drawbacks of the Waterfall model.

Any two:

1. **Changes are difficult to accommodate** once a phase is completed.
2. **Working software is delivered very late**, usually only at the end of the project.

---

### 4. What is meant by incremental development?

**Incremental development** is a software development approach where the system is built and delivered in **small, usable parts (increments)**, with each increment adding new functionality to the previous one.

---

### 5. State any two Agile values.

Any two:

1. **Individuals and interactions over processes and tools**
2. **Working software over comprehensive documentation**

---

### 6. What is a sprint?

A **sprint** is a **fixed time-boxed iteration (usually 1–4 weeks)** in which a Scrum team plans, develops, tests, and delivers a **working increment of software**.

---

### 7. What is meant by "responding to change" in Agile?

"**Responding to change**" means that Agile teams **welcome and adapt to changing requirements**, even late in development, instead of rigidly following an initial plan.

---

### 8. Define stakeholder.

A **stakeholder** is **any individual or group that can affect or is affected by a software product**, such as users, developers, managers, customers, sponsors, or regulatory bodies.

---

### 9. What is the Agile Manifesto?

The **Agile Manifesto** is a set of **four core values and twelve principles** created in 2001 that guide Agile software development by prioritizing **people, working software, customer collaboration, and adaptability**.

---

### 10. What is RAD?

**RAD (Rapid Application Development)** is a software development model that focuses on **fast delivery through prototyping, parallel development, and early user involvement**, making it suitable when requirements are clear and modular.

---

If you want next:

- **5-mark expanded answers**
- **Diagram-based answers**
- **Examiner-preferred wording**
- **MCQs from these questions**

Say the word.

**Sources**

Explain the need for Agile Software Development.

Compare traditional software development and Agile development.

Explain the four Agile values with examples.

Describe the concept of incremental delivery in Agile.

Explain the onion model of stakeholders.

What challenges are faced while implementing Agile?

Explain the role of customer feedback in Agile.

Describe sequential (plan-driven) development models.

Explain why documentation is given less importance in Agile.

Write short notes on:

Waterfall model

Spiral model

Incremental model

Below are **complete, exam-ready 5-mark answers**, written in **clear academic language**, aligned exactly with **BTech evaluation expectations**. You can safely reproduce these in exams.

All answers are derived from your course PDFs

## 1. Explain the need for Agile Software Development.

Traditional software development methods are rigid and assume that requirements remain fixed throughout the project. In reality, customer needs, market conditions, and technologies change frequently. Agile Software Development is needed to address these challenges.

Agile allows software to be developed in **small, manageable iterations**, enabling early delivery of working software. Continuous customer involvement ensures that the product aligns with real user needs. Agile also reduces risk by identifying problems early through frequent testing and feedback. Overall, Agile improves flexibility, customer satisfaction, quality, and time-to-market.

## 2. Compare traditional software development and Agile development.

| Traditional Development | Agile Development |
|---|---|
| Follows a linear, sequential process | Follows an iterative and incremental process |
| Requirements are fixed early | Requirements can change anytime |
| Working software delivered late | Working software delivered frequently |
| Customer involvement is limited | Continuous customer collaboration |
| Change is costly and difficult | Change is welcomed and managed |
| Progress measured by documents | Progress measured by working software |

## 3. Explain the four Agile values with examples.

The Agile Manifesto defines four core values:

1. **Individuals and interactions over processes and tools**
   Example: Direct team communication is preferred over strict tool usage.
2. **Working software over comprehensive documentation**
   Example: A working login feature is more valuable than detailed design documents.
3. **Customer collaboration over contract negotiation**
   Example: Regular feedback meetings instead of rigid contract enforcement.
4. **Responding to change over following a plan**
   Example: Updating features based on market trends rather than sticking to outdated plans.

These values focus on delivering real value efficiently.

## 4. Describe the concept of incremental delivery in Agile.

Incremental delivery in Agile means developing and delivering software in **small, usable pieces** called increments. Each increment adds new functionality to the existing system.

After every iteration or sprint, a working version of the product is delivered and reviewed. This allows users to start using features early and provide feedback. Incremental delivery reduces risk, improves quality, and ensures that the product evolves based on actual customer needs.

---

## 5. Explain the onion model of stakeholders.

The onion model classifies stakeholders based on their **closeness and influence** on the product.

- **Core (Center)**: The product itself.
- **Primary stakeholders**: Users, system administrators, support staff who directly use the system.
- **Secondary stakeholders**: Managers, sales teams, legal teams involved in decisions.
- **Tertiary stakeholders**: Sponsors, regulators, public, and media who influence funding, compliance, and perception.

This model helps identify stakeholder priorities and manage expectations effectively.

---

## 6. What challenges are faced while implementing Agile?

Although Agile is flexible, teams face several challenges:

- Frequent requirement changes can cause confusion and rework.
- Agile requires continuous customer involvement, which may not always be available.
- Communication gaps can arise, especially in distributed teams.
- Self-organizing teams may struggle without skilled or experienced members.
- Continuous testing and frequent delivery can increase workload and pressure.

Proper planning and team maturity are essential to overcome these challenges.

---

## 7. Explain the role of customer feedback in Agile.

Customer feedback is central to Agile development. After each sprint, the team presents a working product to the customer and collects feedback.

This feedback helps:

- Validate requirements
- Identify improvements early
- Adjust priorities for future sprints
- Reduce the risk of building unwanted features

By involving customers continuously, Agile ensures higher satisfaction and better alignment with business needs.

---

## 8. Describe sequential (plan-driven) development models.

Sequential or plan-driven models follow a **step-by-step linear process** where each phase is completed before moving to the next.

Common models include:

- **Waterfall Model**
- **V-Model**
- **Spiral Model**
- **Incremental Model**
- **RAD Model**

These models work well when requirements are clearly defined and unlikely to change. However, they lack flexibility and handle changes poorly.

---

## 9. Explain why documentation is given less importance in Agile.

Agile does not eliminate documentation but **prioritizes working software over excessive documentation**.

Heavy documentation can delay delivery and become outdated quickly. Agile prefers just-enough documentation that supports development. The main focus is on delivering usable features early and improving them through feedback, making documentation secondary to real value delivery.

---

## 10. Short Notes

### (a) Waterfall Model

The **Waterfall Model** is a linear development approach where phases such as requirements, design, implementation, testing, and deployment occur sequentially. Once a phase is completed, it cannot be revisited easily. It is suitable for projects with stable and well-defined requirements.

---

### (b) Spiral Model

The **Spiral Model** combines iterative development with risk analysis. Development progresses through repeated cycles called spirals. Each cycle involves planning, risk analysis, development, and evaluation. It is suitable for large and high-risk projects.

---

### (c) Incremental Model

The **Incremental Model** develops the system in small functional modules. Each increment delivers a working part of the software. This allows early delivery, easier testing, and better handling of changes compared to the Waterfall model.

---

If you want next, I can:

- Convert these into **10-mark answers**
- Add **diagrams where examiners expect them**
- Highlight **most probable exam questions**
- Create **ready-to-memorize bullet answers**

Say what you want next.

**Sources**

Explain Agile Software Development in detail with a real-world example.

Describe the Agile Manifesto and explain all four values.

Explain the 12 principles of Agile Software Development.

Compare Agile and Waterfall models with suitable examples.

Explain stakeholder classification using the onion model with a diagram.

Discuss the limitations of traditional development models and how Agile solves them.

Below are **complete 10-mark, exam-ready answers**, written in **structured academic style**, with **examples and diagram descriptions** exactly how university evaluators expect.

All content is strictly based on your course material
Unit 1

---

# 1. Explain Agile Software Development in detail with a real-world example.

**Agile Software Development** is an **iterative and incremental approach** to building software that focuses on **flexibility, continuous feedback, customer collaboration, and frequent delivery of working software**. Unlike traditional methods where everything is planned upfront, Agile accepts that requirements evolve over time.

In Agile, development is divided into **short iterations called sprints**, usually lasting 1–4 weeks. Each sprint results in a **working increment of software** that can be reviewed by the customer. Based on

feedback, requirements are refined for the next sprint. This reduces risk, improves quality, and ensures alignment with real customer needs.

### Real-world example: Food Delivery App

Instead of building the entire app at once:

- Sprint 1: Login and restaurant browsing
- Sprint 2: Cart and ordering
- Sprint 3: Payment
- Sprint 4: GPS tracking
- Sprint 5: Offers and notifications

After every sprint, users test the app and give feedback. This allows quick corrections and faster release. Agile therefore delivers **value early and continuously**, making it ideal for dynamic markets.

---

## 2. Describe the Agile Manifesto and explain all four values.

The **Agile Manifesto**, created in 2001, defines the core philosophy of Agile development. It consists of **four values** that guide decision-making.

1. **Individuals and interactions over processes and tools**
   Agile emphasizes skilled people and communication rather than rigid tools.
   Example: A team discussion is preferred over strict tool-based reporting.
2. **Working software over comprehensive documentation**
   Delivering a usable product is more valuable than maintaining heavy documents.
   Example: A working payment feature is prioritized over a long design report.
3. **Customer collaboration over contract negotiation**
   Agile promotes regular customer involvement instead of fixed contracts.
   Example: Customers review each sprint instead of waiting for final delivery.
4. **Responding to change over following a plan**
   Agile welcomes change even late in development.
   Example: Adding a new feature based on market demand.

These values help teams deliver **useful, adaptable, and high-quality software**.

---

## 3. Explain the 12 principles of Agile Software Development.

The Agile Manifesto defines **12 principles** that support its values:

1. Customer satisfaction through early and continuous delivery
2. Welcome changing requirements
3. Deliver working software frequently
4. Business people and developers work together daily

5. Build projects around motivated individuals
6. Face-to-face communication is most effective
7. Working software is the primary measure of progress
8. Promote sustainable development pace
9. Continuous attention to technical excellence
10. Simplicity is essential
11. Best designs emerge from self-organizing teams
12. Regular reflection and process improvement

These principles ensure **quality, adaptability, teamwork, and continuous improvement** throughout the project lifecycle.

---

## 4. Compare Agile and Waterfall models with suitable examples.

| Aspect | Waterfall Model | Agile Model |
|---|---|---|
| Development style | Sequential | Iterative |
| Requirement changes | Difficult | Welcomed |
| Delivery | At end | Frequent |
| Customer involvement | Minimal | Continuous |
| Risk handling | High risk | Low risk |
| Progress measure | Documents | Working software |

### Example

Waterfall suits a **banking compliance system** with fixed requirements.
Agile suits a **mobile app** where user preferences change frequently.

Agile is preferred for modern, fast-changing environments.

---

## 5. Explain stakeholder classification using the onion model with a diagram.

The **onion model** classifies stakeholders based on their **closeness to the product**.

### Layers:

- **Center**: Product
- **Primary stakeholders**: Users, system admins, support staff
- **Secondary stakeholders**: Managers, sales, legal teams
- **Tertiary stakeholders**: Sponsors, regulators, public, media

### Diagram description (for exam):

Draw concentric circles like an onion:

- Center circle labeled “Product”
- Next ring: Primary stakeholders
- Next ring: Secondary stakeholders
- Outer ring: Tertiary stakeholders

This model helps prioritize stakeholder influence and manage expectations.

---

## 6. Discuss the limitations of traditional development models and how Agile solves them.

### Limitations of traditional models:

- Rigid planning and fixed requirements
- Late delivery of working software
- Poor customer involvement
- High risk of failure if requirements change
- Difficult and costly changes

### How Agile solves these:

- Iterative development allows flexibility
- Early and frequent delivery reduces risk
- Continuous customer feedback improves relevance
- Change is welcomed and managed
- Quality improves through continuous testing

Agile therefore provides **better adaptability, faster delivery, and higher customer satisfaction**.

---

If you want next, I can:

- Add **exam diagrams (step-by-step drawing instructions)**
- Convert these into **model answers exactly matching university marking schemes**
- Create **1–2 page condensed revision notes**
- Predict **most likely 10-mark questions**

Say the word.

**Sources**

What is Scrum?

Define Product Backlog.

Who is a Product Owner?

What is a Scrum Master?

What is Kanban?

Define WIP limits.

What is Kaizen?

What is XP?

What is a Sprint Review?

What is Definition of Done?

Below are **precise, BTech 2-mark exam-ready answers**. Direct. No fluff. You can write these verbatim.

All definitions are aligned with your course PDFs.

---

### 1. What is Scrum?

**Scrum** is an **Agile framework** that develops software in **short, time-boxed iterations called sprints**, using defined roles, events, and artifacts to deliver **incremental working software**.

---

### 2. Define Product Backlog.

The **Product Backlog** is an **ordered list of features, requirements, enhancements, and fixes** needed for a product, maintained and prioritized by the Product Owner.

---

### 3. Who is a Product Owner?

The **Product Owner** represents the **business and customer perspective**, defines the product vision, prioritizes the Product Backlog, and ensures **maximum business value** is delivered.

---

### 4. What is a Scrum Master?

The **Scrum Master** facilitates the Scrum process, **removes impediments**, coaches the team, and ensures Scrum practices are followed effectively.

---

### 5. What is Kanban?

**Kanban** is an Agile method that uses a **visual workflow board** to manage work in progress, improve flow, and reduce bottlenecks through continuous delivery.

---

### 6. Define WIP limits.

**WIP (Work In Progress) limits** restrict the **maximum number of tasks** allowed in a workflow stage to prevent overload and improve flow efficiency.

---

### 7. What is Kaizen?

**Kaizen** is a **continuous improvement philosophy** that focuses on making **small, incremental improvements** regularly to enhance processes, quality, and efficiency.

---

### 8. What is XP?

**XP (Extreme Programming)** is an Agile methodology that emphasizes **engineering excellence** through practices like **pair programming, test-driven development, continuous integration, and frequent releases**.

---

### 9. What is a Sprint Review?

A **Sprint Review** is a meeting held at the **end of a sprint** where the team demonstrates the working product increment to stakeholders and collects feedback.

---

### 10. What is Definition of Done?

The **Definition of Done (DoD)** is a **shared checklist of criteria** that must be met for a task or user story to be considered complete, such as coding, testing, and integration.

---

If you want next:

- **5-mark Scrum answers**
- **Scrum diagrams**
- **Scrum vs Kanban comparison**
- **MCQs from Scrum & XP**

Say what you want to tackle next.

Explain Scrum roles.

Describe Scrum ceremonies.

Explain Scrum practices.

What are the advantages of Scrum?

Explain Lean Software Development.

Explain Kanban with a workflow diagram.

Describe the five Scrum values.

Explain Extreme Programming and its philosophy.

Write short notes on:

Pair programming

Test Driven Development

When should Scrum be used?

Below are **complete, exam-ready 5-mark answers**, written exactly at **BTech standard**. Clear structure. No filler. You can reproduce these safely.

All content is aligned with your course material.

---

# 1. Explain Scrum roles.

Scrum defines **three clear roles** to ensure accountability and smooth delivery.

1. **Product Owner**
   Represents business and customer needs. Responsible for defining product vision, prioritizing the Product Backlog, and maximizing business value.
2. **Scrum Master**
   Acts as a facilitator and coach. Ensures Scrum principles are followed, removes impediments, and helps the team improve continuously.
3. **Development Team**
   A cross-functional, self-organizing group responsible for designing, coding, testing, and delivering a potentially shippable product increment every sprint.

---

# 2. Describe Scrum ceremonies.

Scrum ceremonies are structured meetings that maintain rhythm and transparency.

1. **Sprint Planning** – Team selects backlog items and defines sprint goal.

2. **Daily Scrum** – 15-minute daily meeting to track progress and identify blockers.
3. **Sprint Review** – Demonstration of completed work to stakeholders for feedback.
4. **Sprint Retrospective** – Team reflects on what went well and what can be improved.

These ceremonies ensure alignment, inspection, and adaptation.

---

## 3. Explain Scrum practices.

Scrum relies on core practices that support Agile delivery:

- **Timeboxing** – Fixed duration sprints and meetings.
- **Empirical process control** – Transparency, inspection, adaptation.
- **Incremental delivery** – Working software delivered every sprint.
- **Prioritized backlog** – Work ordered by business value.
- **Self-organizing teams** – Teams decide how to do the work.
- **Definition of Done** – Common completion criteria.

These practices improve predictability and quality.

---

## 4. What are the advantages of Scrum?

Advantages of Scrum include:

- Faster delivery through short sprints
- High adaptability to changing requirements
- Continuous customer feedback
- Early risk detection
- Improved team collaboration
- Higher product quality due to frequent testing

Scrum is ideal for complex and evolving projects.

---

## 5. Explain Lean Software Development.

Lean Software Development focuses on **delivering maximum value by eliminating waste**.

Key principles include:

- Eliminate non-value-adding activities
- Build quality in
- Deliver fast
- Optimize the whole system

- Continuous improvement (Kaizen)

Lean helps reduce delays, defects, and unnecessary features, improving efficiency and customer satisfaction.

---

## 6. Explain Kanban with a workflow diagram.

**Kanban** is a visual workflow management method that improves flow and limits overload.

### Workflow diagram (draw this in exam):

```
vbnet

To Do  →  In Progress  →  Testing  →  Done
```

Key features:

- Visual board
- WIP limits to control work
- Continuous delivery
- Bottleneck identification

Kanban improves transparency and efficiency.

---

## 7. Describe the five Scrum values.

Scrum is built on five core values:

1. **Commitment** – Dedication to sprint goals
2. **Courage** – Address challenges openly
3. **Focus** – Concentrate on sprint work
4. **Openness** – Transparency in progress and issues
5. **Respect** – Trust and value team members

These values promote strong teamwork and trust.

---

## 8. Explain Extreme Programming and its philosophy.

**Extreme Programming (XP)** is an Agile methodology that takes **best software practices to the extreme**.

Philosophy:
“If a practice is good, do it continuously.”

XP emphasizes:

- Very frequent releases
- Constant testing
- Simple design
- Continuous feedback

XP improves code quality, flexibility, and customer satisfaction.

---

# 9. Short Notes

### (a) Pair Programming

Two developers work at one workstation. One writes code (driver) while the other reviews (navigator). This improves code quality, knowledge sharing, and defect detection.

---

### (b) Test Driven Development (TDD)

TDD follows a **test-first approach**:

1. Write a failing test
2. Write code to pass the test
3. Refactor the code

TDD ensures correctness and reduces defects.

---

# 10. When should Scrum be used?

Scrum should be used when:

- Requirements change frequently
- Fast delivery is required
- Customer feedback is critical
- Project is complex or uncertain
- Cross-functional teams are available

Scrum is not ideal for rigid, fixed-scope projects.

---

If you want next:

- **10-mark Scrum answers**
- **Numericals from Agile metrics**
- **Scrum vs Kanban vs XP comparison**
- **One-night revision notes**

Say what to do next.

Explain Scrum framework in detail with roles, ceremonies, and artifacts.

Describe how Scrum works step by step.

Compare Scrum and Kanban.

Explain Lean principles and waste management in Agile.

Explain Extreme Programming and its 12 practices.

Compare Agile model with RAD model.

Compare Agile model with Iterative Waterfall model.

Explain scaling Agile using SAFe, LeSS, and Scrum of Scrums.

Below are **full 10-mark, exam-ready answers**, written in **structured academic format**, exactly how evaluators expect. Clear headings, comparisons, and examples. You can reproduce these safely.

All content is aligned with your syllabus and PDFs.

---

# 1. Explain Scrum framework in detail with roles, ceremonies, and artifacts.

**Scrum** is an Agile framework used to develop complex products through **iterative and incremental delivery**. It is lightweight, flexible, and focuses on transparency, inspection, and adaptation.

## (a) Scrum Roles

1. **Product Owner**
   Owns the product vision, manages and prioritizes the Product Backlog, and ensures maximum business value.
2. **Scrum Master**
   Facilitates Scrum events, removes impediments, and coaches the team to follow Scrum principles.
3. **Development Team**
   A cross-functional, self-organizing team responsible for delivering a potentially shippable product increment each sprint.

## (b) Scrum Ceremonies

- **Sprint Planning** – Decide sprint goal and select backlog items.

- **Daily Scrum** – 15-minute meeting to track progress and blockers.
- **Sprint Review** – Demonstrate completed work and gather feedback.
- **Sprint Retrospective** – Reflect and improve processes.

### (c) Scrum Artifacts

- **Product Backlog** – Ordered list of all requirements.
- **Sprint Backlog** – Items selected for the current sprint.
- **Product Increment** – Working software delivered at sprint end.

Scrum ensures continuous delivery, adaptability, and high-quality outcomes.

---

## 2. Describe how Scrum works step by step.

1. **Product Vision Creation** by Product Owner
2. **Product Backlog Creation** with prioritized user stories
3. **Sprint Planning** to select work for the sprint
4. **Sprint Execution** involving design, coding, testing, and integration
5. **Daily Scrum** for progress tracking
6. **Sprint Review** to present working increment to stakeholders
7. **Sprint Retrospective** to improve team performance
8. **Increment Release** and backlog refinement
9. **Next Sprint Begins**

This cycle repeats until the product is complete.

---

## 3. Compare Scrum and Kanban.

| Aspect | Scrum | Kanban |
|---|---|---|
| Structure | Time-boxed sprints | Continuous flow |
| Roles | Defined roles | No mandatory roles |
| Planning | Sprint-based | On-demand |
| WIP limits | Implicit | Explicit |
| Change handling | Between sprints | Anytime |
| Delivery | At sprint end | Continuous |

**Scrum** suits projects needing structure, while **Kanban** suits continuous maintenance and support work.

---

## 4. Explain Lean principles and waste management in Agile.

**Lean Software Development** focuses on **maximizing customer value while minimizing waste**.

### Lean Principles:

- Eliminate waste
- Build quality in
- Deliver fast
- Defer commitment
- Respect people
- Optimize the whole system
- Continuous improvement (Kaizen)

### Waste Management:

Lean removes **Muda**, activities that add no value.

Common wastes:

- Unnecessary features
- Waiting time
- Rework and defects
- Excess documentation
- Multitasking
- Over-planning

By eliminating waste, Agile teams improve speed, quality, and efficiency.

---

## 5. Explain Extreme Programming and its 12 practices.

**Extreme Programming (XP)** is an Agile methodology focused on **engineering excellence and customer satisfaction**.

### Philosophy:

“If a practice is good, do it all the time.”

### 12 Practices of XP:

1. Planning Game
2. Small Releases
3. System Metaphor
4. Simple Design
5. Test-Driven Development

6. Refactoring
7. Pair Programming
8. Collective Code Ownership
9. Continuous Integration
10. Sustainable Pace (40-hour week)
11. On-site Customer
12. Coding Standards

XP improves flexibility, code quality, and rapid feedback.

---

## 6. Compare Agile model with RAD model.

| Aspect | Agile Model | RAD Model |
|---|---|---|
| Focus | Incremental delivery | Rapid prototyping |
| Customer feedback | Continuous | Early prototypes |
| Development | Iterative | Prototype-driven |
| Quality | High due to testing | May evolve later |
| Best suited for | Complex, evolving systems | Modular, UI-heavy systems |

Agile delivers **finished features**, while RAD delivers **quick prototypes first**.

---

## 7. Compare Agile model with Iterative Waterfall model.

| Aspect | Agile | Iterative Waterfall |
|---|---|---|
| Flexibility | High | Limited |
| Delivery | Frequent increments | Partial iterations |
| Change handling | Welcomed | Costly |
| Progress measure | Working software | Phase completion |
| Risk | Low | Moderate |

Agile adapts better to changing requirements and reduces failure risk.

---

## 8. Explain scaling Agile using SAFe, LeSS, and Scrum of Scrums.

Scaling Agile is required when **multiple teams work on a large product**.

### (a) Scrum of Scrums

- Coordination between multiple Scrum teams

- Representatives meet to manage dependencies
- Simple and lightweight

### (b) SAFe (Scaled Agile Framework)

- Enterprise-level framework
- Aligns business, development, and operations
- Uses Agile Release Trains (ARTs)
- Suitable for large organizations

### (c) LeSS (Large Scale Scrum)

- Extends Scrum with minimal changes
- One product backlog, one Product Owner
- Focuses on simplicity and transparency

These frameworks help Agile scale without losing flexibility.

---

If you want next:

- **Numerical problems with solved answers**
- **Diagrams you can draw in exams**
- **Most-likely 10-mark questions**
- **One-day crash revision notes**

Tell me what to hit next.

What is Agile estimation?

Define velocity.

What is sprint burndown chart?

Define cycle time.

What is throughput?

What is escape rate?

Define ROI.

What is lead time?

What is defect density?

What is time-to-market?

Below are **crisp, BTech 2-mark, exam-ready definitions**. Direct. No extra theory. You can write these **word-for-word**.

All are aligned with your Agile metrics unit.

---

### 1. What is Agile estimation?

**Agile estimation** is the process of **predicting the effort, time, and cost** required to complete user stories or backlog items using **relative measures** such as story points instead of exact timelines.

---

### 2. Define velocity.

**Velocity** is the **average number of story points completed by a team per sprint**, used to predict future delivery capacity.

---

### 3. What is sprint burndown chart?

A **sprint burndown chart** is a graphical representation that shows the **remaining work versus time** in a sprint, helping track progress toward the sprint goal.

---

### 4. Define cycle time.

**Cycle time** is the **total time taken for a work item** from the moment development starts until it is completed.

---

### 5. What is throughput?

**Throughput** is the **number of work items or user stories completed** by a team in a given time period, usually one sprint.

---

### 6. What is escape rate?

**Escape rate** is the **percentage of committed backlog items that were not completed** during a sprint, indicating predictability issues.

---

### 7. Define ROI.

**ROI (Return on Investment)** measures the **financial benefit of a project compared to its cost**, calculated as:

ROI = (Value Delivered − Cost) / Cost × 100

### 8. What is lead time?

**Lead time** is the **total time taken from a request being made** until the feature is delivered to the customer.

---

### 9. What is defect density?

**Defect density** is the **number of defects per unit size of software**, such as per module or per KLOC, used to measure software quality.

---

### 10. What is time-to-market?

**Time-to-market** is the **total time taken from the start of development until the product or feature is released** to customers.

---

If you want next:

- **Solved numericals from these metrics**
- **5-mark explanations**
- **Graphs + how to draw them in exams**
- **Most common numerical patterns examiners repeat**

Say the word.

Explain Agile metrics and their importance.

Describe team-level Agile metrics.

Describe product-level Agile metrics.

Describe process-level Agile metrics.

Explain Agile planning levels.

Explain sprint-level planning and estimation.

Explain release-level planning.

What is Planning Poker?

Explain Wideband Delphi estimation technique.

Write short notes on:

Burndown chart

Velocity

Below are **complete, exam-ready 5-mark answers**, written in **clean academic language**, exactly at **BTech evaluation level**. Structured, direct, and safe to reproduce.

All content aligns with your Agile metrics and planning unit.

---

## 1. Explain Agile metrics and their importance.

**Agile metrics** are quantitative measures used to **track progress, productivity, quality, and value delivery** in Agile projects. Unlike traditional metrics that focus on documentation and schedules, Agile metrics emphasize **working software, team behavior, and customer value**.

### Importance of Agile metrics:

- Provide **real-time visibility** into progress
- Help in **predicting delivery timelines**
- Identify **bottlenecks and risks early**
- Support **continuous improvement**
- Measure **customer satisfaction and business value**

Agile metrics enable data-driven decisions and improve transparency.

---

## 2. Describe team-level Agile metrics.

**Team-level metrics** focus on team performance and delivery capability.

Key metrics include:

- **Velocity**: Average story points completed per sprint
- **Sprint Burndown**: Remaining work vs time in a sprint
- **Cycle Time**: Time taken to complete a work item
- **Throughput**: Number of items completed per sprint
- **Escape Rate**: Percentage of incomplete committed items
- **Team Morale/Happiness**: Measures engagement and motivation

These metrics help assess **predictability, efficiency, and workload balance**.

---

## 3. Describe product-level Agile metrics.

**Product-level metrics** evaluate the **quality, value, and market impact** of the product.

Common metrics:

- **Time-to-Market**: Speed of product release
- **Release Frequency**: How often new versions are delivered
- **Defects**: Number of bugs found in production
- **Technical Debt**: Pending maintenance and refactoring work
- **Customer Satisfaction**: Feedback and user happiness

These metrics ensure the product delivers **business value and user satisfaction**.

---

## 4. Describe process-level Agile metrics.

**Process-level metrics** assess the **efficiency and effectiveness** of Agile workflows.

Key metrics:

- **Value Delivered**: Total business value of completed work
- **ROI (Return on Investment)**: Value gained vs cost
- **Cost per Iteration**: Expense per sprint
- **Lead Time**: Time from request to delivery
- **Flow Efficiency**: Percentage of active working time

These metrics help optimize processes and **eliminate waste**.

---

## 5. Explain Agile planning levels.

Agile planning occurs at **three levels**, each serving a different purpose:

1. **Project-Level Planning**
   Defines product vision, roadmap, scope, and high-level estimates.
2. **Release-Level Planning**
   Breaks the product backlog into releases and estimates delivery timelines.
3. **Sprint-Level Planning**
   Plans detailed work for a single sprint.

This layered approach ensures **flexibility with predictability**.

---

## 6. Explain sprint-level planning and estimation.

**Sprint-level planning** occurs at the start of each sprint.

Key activities:

- Define **sprint goal**
- Select user stories from Product Backlog
- Break stories into tasks
- Estimate tasks in **hours or days**
- Create Sprint Backlog

Estimation uses **bottom-up approach** and considers team capacity and Definition of Done. It ensures **realistic sprint commitments**.

---

## 7. Explain release-level planning.

**Release-level planning** converts the product vision into **time-boxed releases** containing multiple sprints.

Key points:

- Uses **user stories and story points**
- Considers business priority, team velocity, and capacity
- Estimates number of releases required
- Allows scope adjustment based on feedback

It supports **incremental delivery and early value realization**.

---

## 8. What is Planning Poker?

**Planning Poker** is a **consensus-based Agile estimation technique** used to estimate story points.

Process:

- Team members select estimation cards (Fibonacci series)
- Cards are revealed simultaneously
- Differences are discussed
- Consensus is reached

It reduces bias and improves estimation accuracy.

---

## 9. Explain Wideband Delphi estimation technique.

**Wideband Delphi** is a **collaborative estimation technique** involving multiple experts.

Steps:

1. Experts independently estimate work
2. Estimates are shared anonymously

3. Differences are discussed
4. Revised estimates are made
5. Consensus is achieved

It improves accuracy by leveraging **collective expertise**.

---

# 10. Short Notes

## (a) Burndown Chart

A **burndown chart** visually shows **remaining work versus time** in a sprint or release. It helps track progress and identify delays early.

---

## (b) Velocity

**Velocity** is the **average number of story points completed per sprint**. It helps predict future sprint capacity and plan releases realistically.

---

If you want next:

- **Solved numerical problems**
- **Diagram-based answers**
- **10-mark planning & metrics answers**
- **Last-day revision sheet**

Tell me what to do next.

Explain Agile planning and estimation in detail.

Describe different Agile metrics with examples.

Explain project-level, release-level, and sprint-level planning.

Discuss estimation techniques used in Agile projects.

Explain the importance of Agile metrics in monitoring progress.

Explain how velocity is calculated and used for planning.

Below are **full 10-mark, exam-ready answers**, written in **structured academic style**, aligned exactly with **BTech evaluation expectations**. Clear headings, examples, and no unnecessary filler.

# 1. Explain Agile planning and estimation in detail.

**Agile planning and estimation** is a continuous, iterative process used to forecast **time, effort, cost, and delivery** while accepting uncertainty and change. Unlike traditional upfront planning, Agile plans **progressively**, refining estimates as more information becomes available.

Agile planning focuses on:

- Delivering value early and continuously
- Adapting plans based on feedback and progress
- Using **relative estimation** instead of fixed deadlines

Estimation is done using **user stories** and **story points**, which represent effort, complexity, and risk rather than exact time. Planning is carried out at **multiple levels** to balance flexibility with predictability.

Benefits:

- Reduced risk due to early delivery
- Better handling of changing requirements
- More realistic and reliable forecasts
- Improved transparency and team ownership

---

# 2. Describe different Agile metrics with examples.

Agile metrics are used to **measure progress, quality, productivity, and value delivery**. They are generally divided into three categories.

### (a) Team-level metrics

- **Velocity**: Average story points completed per sprint
  Example: 20 story points per sprint
- **Sprint Burndown**: Remaining work vs time
- **Cycle Time**: Time taken to complete a task
- **Throughput**: Number of items completed per sprint

### (b) Product-level metrics

- **Time-to-Market**: Time from development start to release
- **Defects**: Bugs found in production
- **Release Frequency**: Number of releases per time period
- **Customer Satisfaction**: Feedback and ratings

### (c) Process-level metrics

- **Lead Time**: Request to delivery duration

- **ROI**: Value gained compared to cost
- **Flow Efficiency**: Active work time vs waiting time

These metrics provide **data-driven insights** for improvement.

---

## 3. Explain project-level, release-level, and sprint-level planning.

Agile planning is performed at **three hierarchical levels**.

### (a) Project-level planning

- Defines product vision and roadmap
- Identifies high-level scope, risks, and goals
- Creates and prioritizes the Product Backlog
- Uses rough estimates to forecast timelines and resources

### (b) Release-level planning

- Breaks backlog into **releases**, each containing multiple sprints
- Uses **story points and velocity**
- Determines number of sprints per release
- Allows scope adjustment based on feedback

### (c) Sprint-level planning

- Conducted at the start of each sprint
- Defines sprint goal
- Selects user stories for the sprint
- Breaks stories into tasks
- Estimates tasks in hours or days

This layered approach ensures **strategic alignment and execution clarity**.

---

## 4. Discuss estimation techniques used in Agile projects.

Common Agile estimation techniques include:

1. **Story Points**
   Relative estimation of effort and complexity instead of time.
2. **Planning Poker**
   Team members estimate using Fibonacci cards and reach consensus.
3. **Wideband Delphi**
   Experts independently estimate, discuss differences, and converge.

4. **Estimation by Analogy**
   Estimates based on similar past projects or stories.
5. **Bottom-up Estimation**
   Tasks are estimated individually and aggregated.

These techniques improve **accuracy, collaboration, and realism**.

---

## 5. Explain the importance of Agile metrics in monitoring progress.

Agile metrics are essential for **tracking real progress** and enabling informed decisions.

Importance:

- Measure progress using **working software**, not documents
- Identify delays and bottlenecks early
- Improve predictability and planning accuracy
- Support continuous improvement through retrospectives
- Enhance transparency for stakeholders
- Align delivery with business value

Without metrics, Agile teams operate blindly. Metrics turn intuition into **evidence-based planning**.

---

## 6. Explain how velocity is calculated and used for planning.

### Velocity calculation:

Velocity is the **average number of story points completed per sprint**.

Example:
Sprint 1 = 18 points
Sprint 2 = 22 points
Sprint 3 = 20 points

Velocity = (18 + 22 + 20) / 3 = **20 story points per sprint**

### Use of velocity in planning:

- Predict how much work can be completed in future sprints
- Estimate number of sprints required for a release
- Support realistic sprint commitments
- Improve delivery forecasting

Velocity is not a performance metric. It is a **planning tool**, and should only be compared within the same team.

---

If you want next:

- **All numericals solved step-by-step**
- **One-page cheat sheets for each unit**
- **Diagram-based answers examiners love**
- **Most repeated university questions**

Say what you want to finish with.

A team completes 15, 25, 30, and 20 story points in four sprints. Calculate velocity.

A sprint starts with 40 story points and 27 are completed. Find remaining work.

A team commits to 20 stories but completes 17. Calculate escape rate.

A task starts on Day 5 and finishes on Day 11. Find cycle time.

A project costs ₹1,00,000 and delivers value ₹1,40,000. Calculate ROI.

A system has 20 defects in 5 modules. Find defect density.

Stories with values 20, 30, and 40 are completed. Find value delivered.

A request is made on Day 7 and delivered on Day 21. Find lead time.

Development starts on 10th Jan and release is on 25th Feb. Find time-to-market.

A team completes 18 backlog items in a sprint. Find throughput.

Below are **clean, step-by-step, exam-ready solutions**. Exact formulas. No shortcuts skipped.

---

## 1. Velocity

**Given:** Story points completed = 15, 25, 30, 20
**Number of sprints = 4**

Velocity
= (15 + 25 + 30 + 20) / 4
= 90 / 4
= **22.5 story points per sprint**

---

## 2. Remaining Work (Sprint Burndown)

**Given:**
Total story points = 40
Completed = 27

Remaining work
= 40 − 27
= **13 story points**

---

## 3. Escape Rate

**Given:**
Committed stories = 20
Completed stories = 17

Incomplete stories
= 20 − 17 = 3

Escape Rate
= (3 / 20) × 100
= **15%**

---

## 4. Cycle Time

**Given:**
Start day = Day 5
Finish day = Day 11

Cycle Time
= 11 − 5
= **6 days**

---

## 5. Return on Investment (ROI)

**Given:**
Cost = ₹1,00,000
Value delivered = ₹1,40,000

ROI
= (Value − Cost) / Cost × 100
= (1,40,000 − 1,00,000) / 1,00,000 × 100
= 40,000 / 1,00,000 × 100
= **40%**

---

## 6. Defect Density

**Given:**
Defects = 20

Modules = 5

Defect Density
= 20 / 5
= **4 defects per module**

---

## 7. Value Delivered

**Given:**
Story values = 20, 30, 40

Value Delivered
= 20 + 30 + 40
= **90**

---

## 8. Lead Time

**Given:**
Request day = Day 7
Delivery day = Day 21

Lead Time
= 21 − 7
= **14 days**

---

## 9. Time-to-Market

**Given:**
Start date = 10th Jan
Release date = 25th Feb

Days calculation:

- Jan 10 → Jan 31 = 21 days
- Feb 1 → Feb 25 = 25 days

Time-to-Market
= 21 + 25
= **46 days**

---

## 10. Throughput

**Given:**
Completed backlog items in a sprint = 18

Throughput
= **18 items per sprint**

If you want, I can next:

- Show **how examiners award marks step-wise**
- Give **shortcut tricks** to solve faster
- Create a **numericals-only practice set**
- Predict **most repeated numerical patterns**

Say the word.

1. Differentiate between Scrum vs Kanban ?
2. CompareScrum&Kanbanin terms oftheir approach to task management &workflow ?
3. Create a comprehensive plan for implementing Scrum & Kanban methodologies in your team ? Discuss the steps you take, the challenges you might face & how you would measure success ?
4. Whatareetheactivities & role in Agile Scrum?
5. Describe the Kanban & Crystal in details ?
6. Explain the Agile Scrum framework and how is it different from others ?
7. Explain in continuous integration & version control ?
8. WhatisKanbanandwhen is itneeded ? Discuss the lead time & cycle time in Kanban ?
9. Whatisthedifference between Sprint and Product Backlogs ?
10. List different Agile Prioritizes and Techniques ? Explain any two with proper example ?

Below are **clear, full-length, exam-ready answers** written exactly at **BTech / university evaluation level**. Structured. Direct. No fluff. You can reproduce these safely in exams.

# 1. Differentiate between Scrum and Kanban.

| Aspect | Scrum | Kanban |
|---|---|---|
| Nature | Prescriptive framework | Adaptive method |
| Workflow | Time-boxed sprints | Continuous flow |
| Roles | Product Owner, Scrum Master, Development Team | No mandatory roles |
| Planning | Sprint planning required | Planning on demand |
| Change handling | Changes after sprint | Changes anytime |
| Delivery | At sprint end | Continuous delivery |
| Best suited for | Complex product development | Maintenance and support work |

## 2. Compare Scrum and Kanban in terms of task management and workflow.

**Scrum** manages tasks through a **Sprint Backlog**, where work is committed at the start of a sprint. Tasks remain fixed during the sprint, ensuring focus and predictability. Workflow follows sprint cycles.

**Kanban** manages tasks using a **visual board** with stages such as To Do, In Progress, and Done. Tasks flow continuously without fixed iterations. **WIP limits** control overload and improve efficiency.

Scrum emphasizes **commitment and predictability**, while Kanban emphasizes **flow and flexibility**.

---

## 3. Create a comprehensive plan for implementing Scrum and Kanban in a team. Discuss steps, challenges, and success metrics.

### Implementation Steps

1. Identify project type and team structure
2. Train team on Agile principles
3. Choose Scrum for development projects and Kanban for support work
4. Define workflows, roles, and responsibilities
5. Set up tools like Jira or Trello
6. Start with pilot implementation
7. Review and improve continuously

### Challenges

- Resistance to change
- Lack of Agile experience
- Improper estimation
- Communication gaps
- Overcommitment

### Measuring Success

- Velocity consistency
- Lead time reduction
- Customer satisfaction
- Defect reduction
- On-time delivery

---

## 4. What are the activities and roles in Agile Scrum?

### Roles

- **Product Owner** – Defines vision, prioritizes backlog
- **Scrum Master** – Facilitates process, removes impediments
- **Development Team** – Builds and delivers product increments

### Activities

- Sprint Planning
- Daily Scrum
- Sprint Execution
- Sprint Review
- Sprint Retrospective
- Backlog Refinement

These ensure transparency, inspection, and adaptation.

---

## 5. Describe Kanban and Crystal in detail.

### Kanban

Kanban is a visual workflow management method focused on **continuous delivery and flow optimization**. It uses boards, WIP limits, and pull-based systems to reduce bottlenecks and improve efficiency.

Key principles:

- Visualize work
- Limit WIP
- Manage flow
- Continuous improvement

### Crystal

Crystal is a family of Agile methodologies that adapts practices based on **team size and project criticality**. It emphasizes communication, collaboration, and minimal documentation.

Crystal prioritizes:

- People over processes
- Frequent delivery
- Reflective improvement

---

## 6. Explain the Agile Scrum framework and how it is different from others.

Scrum is a **structured Agile framework** based on **iterations called sprints**. It defines fixed roles, ceremonies, and artifacts to manage complex development.

Differences:

- Scrum is more **rule-driven** than Kanban
- Uses **time-boxing**, unlike continuous flow models
- Strong focus on **roles and accountability**
- Better suited for **product development**

Scrum provides discipline, while other Agile methods emphasize flexibility.

---

## 7. Explain Continuous Integration and Version Control.

### Continuous Integration (CI)

CI is a practice where developers **frequently integrate code changes** into a shared repository. Each integration triggers **automated builds and tests**, detecting issues early and reducing integration risks.

Benefits:

- Faster feedback
- Reduced defects
- Stable codebase

### Version Control

Version control systems track changes to source code, allowing teams to collaborate safely. Examples include Git.

Benefits:

- Change tracking
- Rollback capability
- Parallel development

CI and version control together ensure reliability and speed.

---

## 8. What is Kanban and when is it needed? Discuss lead time and cycle time in Kanban.

**Kanban** is needed when:

- Work arrives continuously
- Requirements change frequently
- Maintenance or support tasks dominate
- Flow optimization is required

### Lead Time

Time from **request to delivery**.

### Cycle Time

Time from **work start to completion**.

Reducing both improves responsiveness and efficiency.

---

## 9. What is the difference between Sprint Backlog and Product Backlog?

| Product Backlog | Sprint Backlog |
|---|---|
| Owned by Product Owner | Owned by Development Team |
| Contains all requirements | Contains sprint-selected items |
| Continuously updated | Fixed during sprint |
| Long-term view | Short-term commitment |

---

## 10. List different Agile prioritization techniques. Explain any two with examples.

### Common Techniques

- MoSCoW
- Kano Model
- Priority Poker
- Value vs Effort
- Stack Ranking

### (a) MoSCoW Technique

Classifies requirements as:

- Must have
- Should have

- Could have
- Won't have

Example: Payment feature = Must have

### (b) Value vs Effort

Features are prioritized based on **business value vs implementation effort**.

Example:
High value + low effort feature is prioritized first.

---

If you want next:

- **10-mark model answers rewritten exactly as per exam rubric**
- **Diagrams you can draw and label**
- **One-night revision cheat sheets**
- **Most repeated university questions**

Say what you want to finish with.

11. Whyuser's story is important in an Agile model ? **How to write a User's story in JIRA ?**
12. Explain working of Agile information system with any one software development example ?
13. Describe Agile Information System ?
14. Differentiate between Product Backlog & Sprint Backlog ?
15. Explain the merit & demerit of Scrum ?
16. **Which are the impacts of Agile process on requirement engineering ?**
17. What is Agile matrices and explain any 5 matrices ?

Below are **exam-ready, long-answer responses** written at **BTech / university standard**. Structured, direct, and safe to reproduce. I'll clearly separate **10-mark** and **5-mark** depth where needed.

---

# 11. Why user stories are important in an Agile model? How to write a user story in JIRA?

### Importance of User Stories in Agile

User stories are the **foundation of Agile requirements engineering**. They describe requirements from the **end-user's perspective**, ensuring that development focuses on **business value instead of technical assumptions**.

**Importance:**

- Keep requirements **simple and understandable**
- Encourage **customer collaboration**
- Support **incremental development**
- Enable **prioritization and estimation**
- Reduce heavy documentation
- Act as the base for sprint planning and testing

A good user story ensures the team builds **what the user actually needs**, not what they assume.

### How to write a User Story in JIRA

A standard user story format is:

> **As a** <type of user>
> **I want** <some goal>
> **So that** <some benefit>

**Steps in JIRA:**

1. Create an **Issue** → Select **Story**
2. Enter the user story summary
3. Add **description** in user story format
4. Define **Acceptance Criteria**
5. Assign **priority**
6. Estimate using **story points**
7. Add to **Product Backlog**

**Example:**

> As a customer
> I want to track my food order
> So that I know when it will arrive

---

## 12. Explain working of Agile Information System with a software development example.

An **Agile Information System (AIS)** is a system that **collects, manages, and communicates Agile project data** such as backlog status, sprint progress, metrics, and feedback.

### Working of Agile Information System

1. Requirements captured as **user stories**

2. Stories stored in **Product Backlog**
3. Sprint planning selects stories into **Sprint Backlog**
4. Progress tracked using **boards and burndown charts**
5. Metrics like velocity and lead time are calculated
6. Feedback updates backlog continuously

### Example: Food Delivery App

- User stories like login, ordering, payment added to backlog
- Sprint 1 delivers login and browsing
- Feedback collected and reflected in next sprint
- AIS tools (JIRA) track tasks, metrics, and progress

AIS enables **transparency, traceability, and continuous improvement**.

---

## 13. Describe Agile Information System.

An **Agile Information System** is a **tool-supported framework** that helps teams **plan, track, measure, and improve Agile projects**.

### Key Components:

- Product Backlog management
- Sprint tracking
- Task boards
- Metrics dashboards
- Reporting and analytics
- Collaboration support

### Benefits:

- Real-time visibility
- Better decision-making
- Improved coordination
- Faster feedback loops
- Accurate progress monitoring

Examples include JIRA, Azure DevOps, Trello.

---

## 14. Differentiate between Product Backlog and Sprint Backlog.

| Product Backlog | Sprint Backlog |
|---|---|
| Contains all requirements | Contains sprint-selected items |
| Owned by Product Owner | Owned by Development Team |
| Dynamic and evolving | Fixed during sprint |
| Long-term planning | Short-term execution |
| Includes features, bugs, enhancements | Includes tasks and stories |

---

# 15. Explain the merits and demerits of Scrum.

### Merits of Scrum

- Faster delivery
- High flexibility
- Continuous customer feedback
- Early risk detection
- Improved team collaboration
- Higher product quality

### Demerits of Scrum

- Requires experienced team
- Difficult in large distributed teams
- Scope creep if not controlled
- Heavy reliance on Product Owner
- Not ideal for fixed-scope projects

---

# 16. Which are the impacts of Agile process on requirements engineering?

Agile **transforms traditional requirements engineering** from upfront documentation to **continuous refinement**.

### Impacts:

- Requirements captured as **user stories**
- Continuous requirement evolution
- Reduced documentation
- Frequent validation through working software

- Strong customer involvement
- Prioritization based on business value

Agile shifts focus from **requirement freezing** to **requirement learning**.

---

## 17. What are Agile metrics? Explain any five metrics.

**Agile metrics** are quantitative measures used to **monitor progress, quality, and value delivery** in Agile projects.

### Five Agile Metrics:

1. **Velocity**
   Average story points completed per sprint.
2. **Sprint Burndown Chart**
   Shows remaining work vs time in a sprint.
3. **Cycle Time**
   Time taken to complete a task after work starts.
4. **Lead Time**
   Time from request to delivery.
5. **Escape Rate**
   Percentage of committed work not completed in a sprint.

These metrics support **planning, predictability, and continuous improvement**.

---

If you want next, I can:

- Rewrite answers in **exact 10-mark word length**
- Provide **diagram instructions examiners expect**
- Create **one-night revision sheets**
- Predict **most likely exam questions**

Just say what you want.

1. Who are the key stakeholders in agile product development? Explain their responsibilities?
2. Explain the different steps involved in Agile retrospectives?
3. How do sprints work in agile?
4. What are the 4 phases of a sprint in agile? Explain in detail each phase?
5. Discuss the principle of scrum planning in detail?
6. What are key phases in extreme programming (XP) lifecycle & How does XP differ from traditional software development methodologies?

7. Write a short note on:
○ i) Continuous integration of Agile
○ ii) Agile Manifesto (4 values explain)
8. Discuss & identify the role played by XP team?
9. How do you monitor & control the Agile products?
10. Explain the difference between Agile & traditional waterfall project management methodologies?

Below are **clear, exam-ready, long-answer responses** written at **BTech / university standard**. Structured, direct, and safe to reproduce. I'll keep the depth appropriate for **8–10 mark questions**.

---

# 1. Who are the key stakeholders in Agile product development? Explain their responsibilities.

Agile product development involves multiple stakeholders, each contributing to value delivery.

### Key stakeholders and responsibilities:

1. **Product Owner**
   - Represents customer and business interests
   - Defines product vision and roadmap
   - Prioritizes the Product Backlog
   - Accepts or rejects completed work
2. **Scrum Master**
   - Facilitates Agile and Scrum practices
   - Removes impediments
   - Coaches the team on Agile principles
   - Ensures continuous improvement
3. **Development Team**
   - Designs, develops, tests, and delivers product increments
   - Self-organizing and cross-functional
   - Responsible for quality and timely delivery
4. **Customers / End Users**
   - Provide requirements and feedback
   - Validate product increments
5. **Sponsors / Management**
   - Provide funding and strategic direction
   - Review progress and outcomes

6. **Other Stakeholders**
    - Legal, operations, regulators, and support teams
    - Influence compliance and deployment

---

## 2. Explain the different steps involved in Agile retrospectives.

An **Agile retrospective** is conducted at the end of a sprint to improve team performance.

### Steps involved:

1. **Set the stage**
   Create a safe environment and define the goal of the retrospective.
2. **Gather data**
   Review sprint outcomes, metrics, successes, and issues.
3. **Generate insights**
   Analyze root causes of problems and identify patterns.
4. **Decide what to do**
   Select actionable improvements for the next sprint.
5. **Close the retrospective**
   Summarize decisions and assign responsibilities.

Retrospectives promote **continuous improvement and learning**.

---

## 3. How do sprints work in Agile?

A **sprint** is a fixed time-boxed iteration, usually **1–4 weeks**, during which a team delivers a working product increment.

### Sprint workflow:

1. Sprint Planning selects backlog items
2. Development, testing, and integration occur
3. Daily Scrum tracks progress
4. Sprint Review demonstrates completed work
5. Sprint Retrospective improves process

Each sprint delivers **potentially shippable software**, ensuring steady progress and adaptability.

---

## 4. What are the four phases of a sprint in Agile? Explain each phase.

### 1. Sprint Planning

The team selects user stories, defines the sprint goal, and estimates tasks.

### 2. Sprint Execution

Design, coding, testing, and integration are performed collaboratively.

### 3. Sprint Review

Completed work is demonstrated to stakeholders, and feedback is collected.

### 4. Sprint Retrospective

The team reflects on performance and identifies improvements.

These phases ensure **planning, execution, validation, and improvement**.

---

## 5. Discuss the principles of Scrum planning in detail.

Scrum planning follows key principles:

- **Time-boxing**: Fixed sprint duration
- **Incremental planning**: Plan only what is needed now
- **Business value prioritization**
- **Team commitment based on capacity**
- **Transparency and collaboration**
- **Adaptation based on feedback**

Scrum planning ensures **realistic commitments and flexibility**.

---

## 6. What are key phases in Extreme Programming (XP) lifecycle? How does XP differ from traditional methodologies?

### XP lifecycle phases:

1. **Planning** – User stories defined and prioritized
2. **Design** – Simple and minimal design
3. **Coding** – Pair programming and continuous integration
4. **Testing** – Test-Driven Development
5. **Release** – Small and frequent releases

### Differences from traditional methods:

| XP | Traditional |
|---|---|
| Iterative | Sequential |

| XP | Traditional |
|---|---|
| Continuous testing | Testing at end |
| Customer involvement | Limited customer role |
| Simple design | Heavy upfront design |
| Frequent releases | Late delivery |

XP focuses on **engineering excellence and rapid feedback**.

---

# 7. Short notes

### (i) Continuous Integration in Agile

Continuous Integration is the practice of **frequently merging code changes** into a shared repository, followed by automated builds and tests. It helps detect defects early and maintains a stable codebase.

### (ii) Agile Manifesto – Four Values

1. Individuals and interactions over processes and tools
2. Working software over comprehensive documentation
3. Customer collaboration over contract negotiation
4. Responding to change over following a plan

These values guide Agile decision-making.

---

# 8. Discuss and identify the role played by the XP team.

The XP team consists of:

- **Customer** – Defines and prioritizes user stories
- **Developers** – Write code, tests, and refactor
- **Coach** – Guides XP practices
- **Tester** – Supports automated and acceptance testing
- **Tracker** – Monitors progress
- **Manager** – Provides resources

XP teams emphasize **collaboration, quality, and rapid feedback**.

---

# 9. How do you monitor and control Agile products?

Agile products are monitored using:

- Sprint reviews and demos

- Daily Scrum meetings
- Agile metrics (velocity, burndown, lead time)
- Task boards and dashboards
- Customer feedback
- Retrospectives

Control is achieved through **inspection, adaptation, and transparency** rather than rigid plans.

---

## 10. Explain the difference between Agile and traditional Waterfall project management.

| Agile | Waterfall |
|---|---|
| Iterative and incremental | Sequential |
| Flexible requirements | Fixed requirements |
| Continuous delivery | Single final delivery |
| High customer involvement | Minimal involvement |
| Change welcomed | Change costly |
| Progress via working software | Progress via documents |

Agile suits **dynamic environments**, while Waterfall suits **stable, well-defined projects**.

---

If you want, I can:

- Rewrite these to **exact 10-mark word limits**
- Add **diagram instructions examiners expect**
- Convert everything into **last-day revision notes**
- Predict **most likely exam questions**

Say the word and we'll finish strong.

11. Define continuous integration & importance of Agile practice?
12. Illustrate with example how XP improves software quality? Discuss at least 2 practices of XP in your answer?
13. Discuss role of scrum master in scrum team? How do these roles contribute to the success of an agile project?
14. Analyze the impact of continuous feedback in Agile methodologies on the SDLC?
15. Evaluate the integration of Agile practices into the traditional software development environment dealing with the organizational changes needed & potential changes faced during the transition? (20 marks)
16. Discuss the various principles of Agile methodologies?

17. Suppose you have to develop a software using Agile models. The software is to be released in increments & development time is fixed to 2 weeks, which is 2 weeks less than defined for any sprint. Discuss how the agile model can be modified to make delivery possible in 2 weeks considering some level of independence between development activities?

Below are **exam-ready, long-answer responses** written at **BTech / university standard**. Clear structure, correct depth, and aligned with typical **10–20 mark evaluation**. You can reproduce these safely.

---

# 11. Define Continuous Integration and explain its importance in Agile practice.

**Continuous Integration (CI)** is an Agile engineering practice where developers **frequently integrate code changes** into a shared repository. Each integration is verified by **automated builds and automated tests** to detect errors early.

### Importance of CI in Agile:

- Detects integration defects early
- Reduces last-minute merge conflicts
- Maintains a stable and working codebase
- Supports frequent releases
- Improves software quality
- Enables fast feedback to developers

CI aligns with Agile principles by ensuring **working software at all times** and supporting rapid, incremental delivery.

---

# 12. Illustrate with an example how XP improves software quality. Discuss at least two XP practices.

**Extreme Programming (XP)** improves software quality by enforcing **strong engineering discipline** and continuous feedback.

### Example: Online Banking System

In an online banking application, frequent changes in transaction rules and security require high reliability.

### XP Practices that improve quality:

#### 1. Test-Driven Development (TDD)

- Tests are written **before** code
- Code is written only to pass tests
- Reduces defects and ensures correctness

Example:
Before implementing fund transfer, tests validate balance checks and transaction limits.

**2. Pair Programming**

- Two developers work together
- One codes, the other reviews continuously
- Reduces bugs and improves design quality

Result:
Early defect detection, shared knowledge, and cleaner code.

Together, these practices ensure **high-quality, maintainable, and defect-resistant software**.

---

## 13. Discuss the role of Scrum Master in a Scrum team. How does this role contribute to Agile project success?

The **Scrum Master** is a **servant leader and facilitator** who ensures Scrum is understood and followed.

### Responsibilities:

- Facilitates Scrum ceremonies
- Removes impediments
- Coaches team on Agile principles
- Protects team from external disruptions
- Encourages continuous improvement
- Ensures transparency and collaboration

### Contribution to success:

- Improves team productivity
- Reduces delays caused by blockers
- Maintains Agile discipline
- Enhances communication
- Enables self-organization

A strong Scrum Master directly impacts **delivery speed, quality, and team morale**.

---

## 14. Analyze the impact of continuous feedback in Agile methodologies on the SDLC.

Continuous feedback is central to Agile and affects **every phase of the SDLC**.

### Impact on SDLC phases:

- **Requirements**: Requirements evolve based on feedback instead of being frozen
- **Design**: Design improves incrementally, avoiding over-engineering
- **Development**: Early feedback reduces rework
- **Testing**: Continuous testing improves defect detection
- **Deployment**: Frequent releases align product with user needs
- **Maintenance**: Feedback guides improvements and optimizations

Overall impact:

- Reduced risk
- Higher customer satisfaction
- Faster delivery
- Better alignment with business goals

Continuous feedback transforms SDLC from a rigid pipeline into an **adaptive learning cycle**.

---

## 15. Evaluate the integration of Agile practices into a traditional software development environment.

(Organizational changes and challenges faced during transition) **[20 Marks]**

Integrating Agile into a traditional environment requires **significant organizational, cultural, and process changes**.

### Organizational changes required:

- Shift from command-and-control to **collaborative leadership**
- Empowerment of self-organizing teams
- Increased customer involvement
- Cross-functional team structure
- Reduced emphasis on heavy documentation

### Process changes:

- From phase-based delivery to incremental delivery
- From fixed requirements to evolving requirements

- From milestone tracking to value tracking

### Challenges faced:

- Resistance to change from management and teams
- Lack of Agile skills and training
- Difficulty in adapting fixed contracts
- Misalignment with existing performance metrics
- Cultural mismatch with Agile values

### Managing the transition:

- Start with pilot projects
- Provide Agile coaching and training
- Gradual adoption instead of forced transformation
- Align metrics with Agile goals

### Evaluation:

Organizations that successfully integrate Agile gain:

- Faster delivery
- Better quality
- Higher customer satisfaction

However, success depends on **mindset change, leadership support, and cultural readiness**.

---

## 16. Discuss the various principles of Agile methodologies.

Agile methodologies are guided by **12 core principles**, including:

- Early and continuous delivery of value
- Welcoming changing requirements
- Frequent delivery of working software
- Close collaboration between business and developers
- Motivated and empowered teams
- Face-to-face communication
- Working software as primary progress measure
- Sustainable development pace
- Technical excellence
- Simplicity
- Self-organizing teams

- Regular reflection and improvement

These principles ensure **flexibility, quality, and continuous improvement**.

---

## 17. Agile delivery in 2-week fixed development time with incremental releases. Discuss model modification.

Given:

- Software released in **increments**
- Development time fixed to **2 weeks**
- Sprint duration is **shorter than usual**
- Activities have some independence

### Modifications to Agile model:

1. **Smaller sprint scope**
   - Reduce user story size
   - Focus on high-priority features only
2. **Parallel development activities**
   - Design, coding, and testing overlap
   - Independent modules developed simultaneously
3. **Stronger CI/CD**
   - Automated testing and deployment
   - Faster integration cycles
4. **Reduced documentation**
   - Focus on essential artifacts only
5. **Feature slicing**
   - Deliver minimal but usable functionality
6. **XP practices**
   - Pair programming and TDD to reduce rework

### Result:

- Delivery possible within 2 weeks
- Reduced risk
- Faster feedback
- Maintained quality despite tight timeline

Agile's flexibility allows **adaptation of sprint structure without breaking core principles**.

---

If you want:

- These rewritten to **exact 20-mark word count**
- **Diagrams + drawing instructions**
- **Last-day ultra-condensed notes**
- **Model answers with examiner keywords highlighted**

Say the word.

18. How does the Agile methodology relate itself in today's software engineering market? Describe the working of Scrum?
19. Describe the challenges of the Agile process?
20. Explain various Agile development methods with examples?
21. Compare Scrum, XP & general Agile teams?
22. Explain Scrum planning principles?
23. What are the advantages & limitations of XP?
24. Describe the 12 main Agile principles?
25. What are the challenges of migrating to Agile methodologies?
26. Explain Scrum with its advantages & disadvantages?

Below are **exam-ready, long-answer responses** written at **BTech / university standard**. Clear structure, correct depth, and safe to reproduce. I've balanced content for **8–10 mark questions** unless explicitly broader.

---

# 18. How does Agile methodology relate to today's software engineering market? Describe the working of Scrum.

## Relevance of Agile in today's market

Modern software markets are **fast-changing, customer-driven, and competitive**. Requirements evolve due to market trends, user feedback, and technology changes. Agile fits this environment because it:

- Supports **rapid and frequent delivery**
- Welcomes changing requirements
- Emphasizes customer collaboration
- Reduces risk through early feedback
- Improves time-to-market

Agile is widely adopted in startups, product companies, SaaS platforms, and even large enterprises.

## Working of Scrum

Scrum works through **short, iterative cycles called sprints**.

Steps:

1. Product Owner creates and prioritizes Product Backlog
2. Sprint Planning selects items for the sprint
3. Sprint Execution involves design, coding, testing
4. Daily Scrum tracks progress and blockers
5. Sprint Review demonstrates the product increment
6. Sprint Retrospective improves process

This cycle repeats until product completion, ensuring continuous value delivery.

---

## 19. Describe the challenges of the Agile process.

Agile adoption involves several challenges:

- Frequent requirement changes may cause rework
- Requires high customer involvement
- Self-organizing teams need skilled members
- Difficult to scale without proper frameworks
- Estimation may be inaccurate initially
- Documentation may be insufficient for compliance-heavy projects
- Cultural resistance in traditional organizations

Despite challenges, Agile succeeds when teams are trained and supported.

---

## 20. Explain various Agile development methods with examples.

### 1. Scrum

Uses time-boxed sprints with defined roles.
Example: Developing a mobile app with 2-week sprints.

### 2. Extreme Programming (XP)

Focuses on engineering excellence.
Example: Financial systems requiring high code quality.

### 3. Kanban

Visualizes workflow and limits WIP.
Example: IT support or maintenance teams.

### 4. Lean Software Development

Eliminates waste and optimizes flow.
Example: Startups focusing on MVP delivery.

### 5. Crystal

Tailors process based on team size and criticality.
Example: Small teams with changing requirements.

---

## 21. Compare Scrum, XP, and general Agile teams.

| Aspect | Scrum | XP | General Agile |
|---|---|---|---|
| Focus | Management framework | Engineering practices | Agile mindset |
| Iterations | Sprints | Short iterations | Varies |
| Practices | Planning, reviews | TDD, pair programming | Flexible |
| Customer involvement | Regular | Continuous | Depends |
| Best suited for | Product development | Quality-critical systems | Mixed projects |

---

## 22. Explain Scrum planning principles.

Scrum planning follows these principles:

- Time-boxed planning
- Incremental and adaptive planning
- Business value prioritization
- Team-based estimation
- Capacity-based commitment
- Transparency and collaboration
- Continuous refinement

These principles ensure realistic delivery and adaptability.

---

## 23. What are the advantages and limitations of XP?

### Advantages of XP

- High software quality
- Early defect detection
- Strong customer involvement

- Continuous testing and integration
- Reduced rework

### Limitations of XP

- Requires disciplined teams
- Heavy customer involvement needed
- Difficult for large distributed teams
- Less suitable for fixed-scope projects

XP is ideal when quality and adaptability are critical.

---

## 24. Describe the 12 main Agile principles.

1. Customer satisfaction through early delivery
2. Welcome changing requirements
3. Frequent delivery of working software
4. Business and developers work together
5. Motivated individuals
6. Face-to-face communication
7. Working software as progress measure
8. Sustainable development pace
9. Technical excellence
10. Simplicity
11. Self-organizing teams
12. Regular reflection and improvement

---

## 25. What are the challenges of migrating to Agile methodologies?

- Resistance to cultural change
- Lack of Agile knowledge and training
- Management mindset shift required
- Existing contract and governance models
- Tooling and process mismatch
- Measuring performance incorrectly
- Scaling Agile across teams

Migration succeeds with leadership support, training, and phased adoption.

---

# 26. Explain Scrum with its advantages and disadvantages.

## Scrum

Scrum is an Agile framework using **sprints, defined roles, and ceremonies** to manage complex development.

## Advantages

- Faster delivery
- High adaptability
- Improved collaboration
- Early risk detection
- Customer satisfaction

## Disadvantages

- Requires experienced team
- Scope creep risk
- Difficult for large distributed teams
- Heavy reliance on Product Owner

Scrum works best for **complex, evolving products**.

---

If you want, I can now:

- Convert these into **exact 10-mark word limits**
- Create **diagram-based answers**
- Give **one-night revision sheets**
- Highlight **most repeated exam questions**

Just tell me how you want to wrap this up.